

God damnit buun, there's no binaries on your site: A Pragmatic Guide to Local Agentic LLMs

I have been getting so wrapped up in testing models, engines, harnesses, and everything else out there LLM-related lately. It's never-ending. But it occurred to me; I haven't actually looked at how you get from:

1. An X post sounding like a fun idea to try this weekend at home, to
2. Actually having something working on your computer that's legitimately useful.

So how do you do it?

Let's take buun's fork of llama.cpp for example. You might have seen it on X recently when Clem posted, asking "is this useful?". I am personally fortunate enough to work with buun on a nearly daily basis, but what if I didn't? I hope he doesn't hate me.

God damnit buun, there's no binaries on your site.

Getting There

The truth is that I don't want to go and build it for a bunch of different systems I don't have, or spin up VMs to test compilation flags and set up virtual environments. I hate all the Linux sysadmin stuff like the rest of us. So I had an agent build it for me. Same as you probably would. But if you don't have that luxury, here's how to build it for free:

The repo has `.github/workflows/build-cuda-windows.yml`. It's `workflow_dispatch` — manual trigger only — and it runs on GitHub's own windows-2022 runners. So Microsoft compiles it, on Microsoft's hardware, for free.

The matrix builds three targets: CUDA 12.4 x64, CUDA 13.3 x64, and CUDA 13.4 arm64. One catch: the workflow uploads nothing. The only path: line in it is commented out. It compiles, proves it compiles, and throws the binary away.

Fixing that is an upload-artifact step — a few lines in your own fork:

YAML

```
- uses: actions/upload-artifact@v4
  with:
    name: llama-windows-cuda-${{ matrix.cuda }}-${{ matrix.arch }}
    path: build/bin/Release/
```

Then, just run a few GitHub CLI commands:

BASH

```
gh repo fork spiritbuun/buun-llama-cpp --clone
# add the step above to .github/workflows/build-cuda-windows.yml, push
gh workflow run "CI (CUDA, windows)" --repo <you>/buun-llama-cpp
gh run watch
gh run download    # your .exe files
```

Worth saying plainly: make-release.yml, release.yml and even winget.yml are all sitting in that directory too, inherited and never fired. The distance between this fork and shipping Windows binaries is closer to a tag than a project.

Note, this is read off the workflow file, I haven't run it myself. Claude says it'll work and I believe it'll probably work fine. I also suggested to buun that he consider providing some binaries on the repo.

The Math (And Why You Shouldn't Have to Do It)

Now that we've avoided that footgun.

Now that we have a working binary on our computer, and assuming our tinkerer has spent the requisite 3 am nights searching reddit for what quantization is, we can talk models.

Qwen 3.8 27B. You already knew. It's the workhorse model everyone has been turning to for generations now on 16GB cards, pushing toe-to-toe with frontier-level models in agentic work. In fact, it scores a massive 46.8 on the Artificial Analysis Agentic Index, performing better than 80% of models compared. A modern miracle.



(Screenshot from Artificial Analysis on Sep. 5th 2026)

But this is where I suspect most people start getting REALLY confused, and rightly so. Qwen3.8-27B ships in 14 weight quants. buun's fork offers 8 KV codecs. That's 112 combinations, before you pick a context length. 56 of them fit on a 16 GiB card with at least 4k of context.

And the ranges are wild enough that the choice genuinely matters. Note that the table below varies *both* the weight quantization and the KV codec together:

Weights	KV Codec	Context Length
GSQ IQ2_XS	f16	96,245
GSQ IQ2_XS	turbo4	373,316
GSQ IQ3_XXS	f16	71,831
GSQ IQ3_XXS	turbo4	278,619
UD-Q3_K_XL	f16	30,632
UD-IQ4_XS	f16	12,322

Same card, same model. 12,322 tokens to 373,316 — a 30× spread depending on two flags a newcomer has no basis for choosing between. And every one of those numbers requires arithmetic you have to do yourself. A dense reading might make you assume 256 KiB/token, which is wrong by 4×, because nothing tells you that only 16 of the 65 layers actually carry KV until you map it out.

Holy shit. That's an insane number of things to track.

Historically, I'd settled on using turbo8/turbo4 for KV, respectively. Which still required doing a lot of manual fiddling to get it maximized. And then you're flatly compressing the entire KV cache without giving any consideration to the sensitivity of the layers. But our newcomer doesn't care about that, cause Clem said there's VBR on X right?

Yes. Yes there is. That complex layer arithmetic? That is exactly the arithmetic VBR is doing for you so you don't have to.

What does this mean? It means I pick the model that I know can get real work done. Thankfully, I've already done months of testing, and nowadays can say confidently that a 3-bit model can legitimately do real work. Remember all those nights reading benchmark charts? Paid off. I settled on GSQ IQ3_XXS.

3-bit model in hand, let's do some work. Engage.

If your agent doesn't respond to that wake word, there are a few different ways to load a model with the binary depending on your OS. To make this easier, I suggest just using a startup script, that way you don't have to remember anything to launch your model server. Our goal is to have a server that we can interface with using a harness. Giddy up.

Here is a launch script for your llama server binary. Just edit the filenames and paths to match your system, make it executable, and launch it from your terminal.

```
#!/bin/bash
MODEL="/path/Qwen3.8-27B-GSQ-RCO-IQ3_XXS-mtp.gguf"
SERVER="/path/buun-llama-cpp/build/bin/llama-server"

# Fork binaries link their own libggml. Without this you may silently load the
# system llama.cpp's libraries and wonder why the turbo types don't exist.
export LD_LIBRARY_PATH="$(dirname "$SERVER"):$LD_LIBRARY_PATH"

$SERVER -m "$MODEL" \
    -ngl 99 \
    -c 262144 \
    -fa on \
    --kv-unified \
    -np 1 \
    -ctk vbr -ctv vbr \
    --vbr-floor t4 \
    -b 2048 -ub 512 \
    --jinja \
```

```
--host 127.0.0.1 --port 8080 \  
--spec-type draft-mtp --spec-draft-n-max 2  
# --mmproj "/path/to/mmproj-F16.gguf" # add for vision (note:  
--mmproj-gpu-swap if it won't fit)
```

`-ngl 99` — all layers on GPU

`-c 262144` — omit entirely to let `--fit` choose one that fits

`-fa on / --kv-unified` — required for VBR, no fallback path

`-np 1` — VBR needs `n_stream == 1`

`--vbr-floor t4` — how bad it's allowed to get, not how it starts

`--spec-draft-n-max 2` — measured 1.68× on this model

Download: <https://gist.github.com/apollo-mg/7f2ba29afe217e056fa1c3621636a559>

Realistic Expectations

So setting realistic expectations is a little tough for me, because I test these things every day. I wanted to try and look at it from the perspective of someone fairly capable of figuring things out, so I don't think getting to this point is asking too much. I do hope it gets much more approachable, and I know there are other options like Unsloth Studio and ollama, but neither of those options currently give you access to turboquant KV cache codecs. TurboQuant KV allows you to squeeze the most out of your context with advanced quantization and fancy math, better than the original codecs currently shipping with standard llama.cpp. But now there's something even better.

VBR.

VBR: Variable Bit Rate

If you've ever done any work with media compression, like video or music compression techniques like MP3 or Divx, you'll probably be aware of something called variable bit rate. Basically, it allows the algorithm to apply more compression in static parts of the file, and less compression where fidelity matters most.

Same idea, except it's varying across the model's layers and the life of your conversation. The first tokens are uncompressed. It only starts spending fidelity when it has to.

I measured the actual number. You'll get about 84,000 tokens of fully lossless, f16 quality on a 16 GB card with the 3-bit model we tested. Concretely, that 84,000 figure comes from taking the

KV budget and dividing it by 64 KiB per token. That number isn't immediately obvious because Qwen3.8-27B is a hybrid architecture where only 16 of its 65 layers actually carry KV. A naive reading assuming 256 KiB/token is off by 4×—and calculating that exact layer arithmetic automatically is the whole reason VBR exists.

Crucially, 84,000 tokens is just the lossless range, not your max context ceiling. With mainline q4_0 (4.5 bpv), every token from the very first one is compressed and degraded, meaning a short 3k-token chat suffers the same compression penalty as a massive 250k-token session. With VBR floor t4, you get pristine f16 quality through ~84k tokens, and it only begins to degrade as the memory budget binds—stretching all the way to that same 250k ceiling while keeping shorter sessions completely lossless. VBR eliminates the need to do capacity math or guess how deep a session will go before starting.

Note, these estimates were taken on my specific system. Other factors will affect your usable memory and context quality based on things like whether you're running a desktop, other models, and video intensive applications.

By default, VBR floors at 1.25 bpv, which allows the most aggressive compression possible for the longest context. But the startup script below raises the floor explicitly to t4 (4.125 bpv), because an agentic harness that autocompacts its own context cares more about fidelity than absolute maximum length.

The Need For Speed?

Thankfully, local inference has gotten a lot better in 2026. The models have not only improved by a significant margin in agentic work, but performance has also never been better. This year brought the introduction of MTP acceleration, or Multi-Token-Prediction to the mainstream. This typically boosts performance by anywhere from 1.3-1.9x depending on workload. We'll be using this in our example today, and it's just a simple flag in the launch script.

Again, a full explanation of MTP is outside the scope of the article, but in a nutshell, MTP uses a tiny part of the model called a draft head. This is like a tiny model itself, and its only job is to predict the next few tokens. The primary model verifies all the drafted tokens in one forward pass, in parallel, and keeps the longest correct prefix. That's the whole reason it's a win: verifying 3 tokens costs about the same as generating 1, because decode is memory-bandwidth-bound, not compute-bound. Reading the weights once to check three guesses is nearly free; reading them three times isn't. Basically like free work.

Not entirely free, but worth it. On this model, I measured a boost of 1.68x.

So how much performance should you actually expect? As you almost certainly guessed, it's entirely hardware dependent. Newer GPUs with faster, more efficient cores are simply better at

this work than older parts. That doesn't mean you need an amazingly powerful card to do real work though. Here's the tokens per second rating for a couple popular cards running this model:

GPU	Backend	pp512 t/s	tg128 t/s	user
RTX 5090	CUDA 13.3	3672 ± 338	104.93 ± 0.40	thetom
RX 9070 XT	Vulkan	795.7 ± 0.6	36.41 ± 0.07	"
RX 9070 XT	HIP	975.50 ± 34	29.93 ± 0.03	me

Harness Selection

There are a lot of options out there in mid 2026 for a harness to choose from. Many are coding focused, few aim to give a complete agentic desktop experience. Personally, one of my favorite options is Hermes Agent. While I don't use it on a daily basis nearly to the extent that one can, I have enough experience with it to know it'll fit most people's needs, right out of the box. So it's a natural fit for our experiment.

Hermes Agent

[Hermes Agent](#) | [Hermes Desktop](#)

DOCS

HERMES-AGENT.NOUSRESEARCH.COM

DISCORD

LICENSE

MIT

BUILT BY

NOUS RESEARCH

LANG

中文

LANG

العربية

LANG

ESPAÑOL

The self-improving AI agent built by [Nous Research](#). It's the only agent with a built-in learning loop — it creates skills from experience, improves them during use, nudges itself to persist knowledge, searches its own past conversations, and builds a deepening model of who you are across sessions. Run it on a \$5 VPS, a GPU cluster, or serverless infrastructure that costs nearly nothing when idle. It's not tied to your laptop — talk to it from Telegram while it works on a cloud VM.

Use any model you want — [Nous Portal](#), OpenRouter, OpenAI, your own endpoint, and [many others](#). Switch with `hermes model` — no code changes, no lock-in.

A real terminal interface	Full TUI with multiline editing, slash-command autocomplete, conversation history, interrupt-and-redirect, and streaming tool output.
Lives where you do	Telegram, Discord, Slack, WhatsApp, Signal, and CLI — all from a single gateway process. Voice memo transcription, cross-platform conversation continuity.
A closed learning loop	Agent-curated memory with periodic nudges. Autonomous skill creation after complex tasks. Skills self-improve during use. FTSS session search with LLM summarization for cross-session recall. Honcho dialectic user modeling. Compatible with the agentskills.io open standard.
Scheduled automations	Built-in cron scheduler with delivery to any platform. Daily reports, nightly backups, weekly audits — all in natural language, running unattended.
Delegates and parallelizes	Spawn isolated subagents for parallel workstreams. Write Python scripts that call tools via RPC, collapsing multi-step pipelines into zero-context-cost turns.
Runs anywhere, not just your laptop	Seven terminal backends — local, Docker, SSH, Singularity, Modal, Daytona, and Vercel Sandbox. Daytona and Modal offer serverless persistence — your agent's environment hibernates when idle and wakes on demand, costing nearly nothing between sessions. Run it on a \$5 VPS or a GPU cluster.
Research-ready	Batch trajectory generation, trajectory compression for training the next generation of tool-calling models.

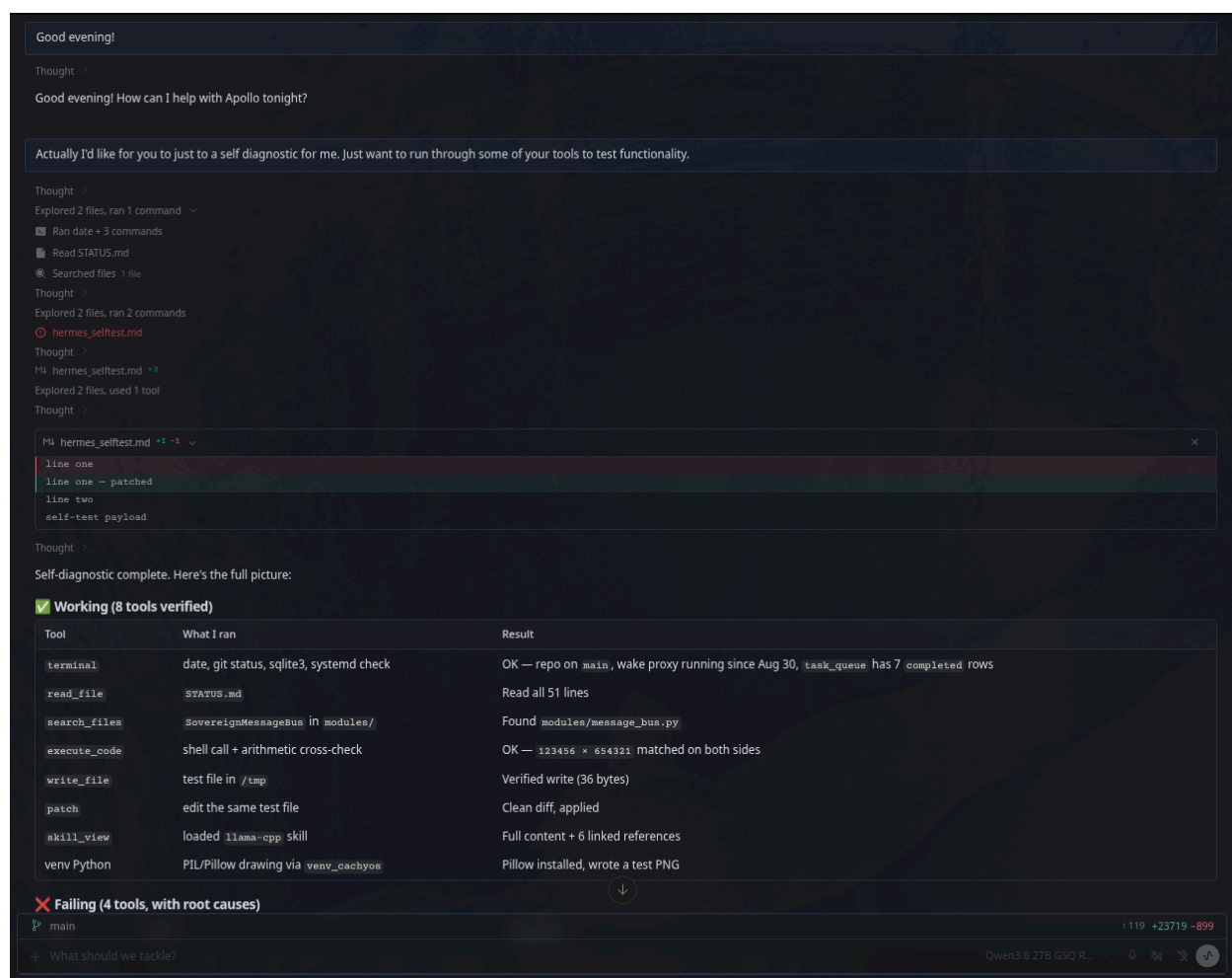
As you can see, it offers just about everything a tinkerer getting into local inference might want. It's easy to talk-to using platforms you already have such as Discord and WhatsApp. It's got Google integration for personal life management. Can run programs, write files, patch, edit. You

get the picture. It really does accomplish a lot of what the premium services like Claude Code and Codex do well, without quite the simplicity.

Setting up Hermes Agent in CachyOS was a relatively straightforward endeavor. Just ran the single line installer (after having my frontier agent inspect the script ahead of time for vulnerabilities) in Konsole, and went through the self-guided configurator.

Most of the setup is just choosing which options you want to enable, such as how you want to talk to your agents (Discord, WhatsApp, email). The most important part is that you choose to connect your Hermes to your custom endpoint, which is your llama-server address (which is found in the startup script). I chose Auto for the type, even though I know mine is specifically OpenAI compatible, because I knew most people will err on the side of caution. Which, incidentally, worked fine for me.

After reading through the documentation and setting up WhatsApp and Discord functionality, I also installed Hermes WebUI (pretty UI, <https://github.com/nesquena/hermes-webui>) and Hermes Desktop. I think the Discord setup was the most challenging for me, but I also knew very little about Discord bots coming into this, so it may be more intuitive for others. Nonetheless, I did get it working with some fiddling.



Limitations and Pitfalls

Before you torch your API keys, there are some hard realities to acknowledge about this setup:

The VBR Tax & Honest Cost: VBR is a massive quality-of-life win, but it has genuine operational costs. It is not free: it requires Flash Attention (-fa on) and unified KV caching (--kv-unified), and it currently exists only in one specific fork with no official binaries. Furthermore, its own diagnostic readout (/props kv_bpv) misreports the underlying math. ***Edit: I previously stated it is incompatible with (-np > 1), that was incorrect. Additionally, on the misreporting /props, buun says that setting --floor-bpv that /props still shows the old defaults.

You Don't Always Need It: For simple coding sessions or conversations that never pass 30,000 tokens, standard mainline f16 fits entirely in VRAM. Mainline f16 works seamlessly today in tools like Ollama without any extra setup, so if your context needs are modest, neither VBR nor static quantization is necessary.

The AMD Reality: The strongest argument for using buun's fork over mainline isn't just efficiency—it's stability. In testing on an RX 9070 XT ([gfx1201](#)), mainline quantized KV entirely collapsed on this model class. Buun's codecs handled the architecture flawlessly. ROCm still arrives late and often broken, leaving the community to fix it. RDNA4 support for turboquant's MMQ path doesn't exist — the config table has zero entries where CDNA has eight — and the cmake flag VBR needs makes upstream crash outright on gfx1201. Meanwhile Vulkan, which nobody had to hand-tune per architecture, quietly beats ROCm on decode. They never learn.

Honesty on Quality: Comparing bitrates directly (e.g., [q4_0](#) at 4.5 bpv vs. [t4](#) at 4.125 bpv) does not directly reflect fidelity. [q4_0](#) relies on a fixed, uncalibrated grid, whereas [t4](#) uses FWHT rotation with measured centroids and per-layer pricing. Because KLD (Kullback-Leibler Divergence) has not been formally measured on this specific model yet, treat this as a structural design difference rather than a proven, mathematical quality win.

The 3-Bit Reality Check

I actually ran a self-diagnostic trace with Hermes Agent while writing this article, using Qwen 3.8 27B squashed all the way down to a 3-bpw quant (IQ3_XXS).

Historically, the assumption has been that crushing a model down to 3 bits lobotomizes its logic. Sure, it might still write a decent email, but ask it to orchestrate an autonomous tool-calling loop and it will inevitably trip over its own shoelaces, hallucinate a file, and crash.

Not anymore. Look at how Qwen handled a blind, multi-tool diagnostic run. I asked it to just "test its tools," and it immediately built an execution plan, batching independent tasks and isolating dependencies:

```
"Watch out for parallel execution: the vision test needs the image path to exist first. Generate the image with terminal, then vision_analyze it. That's a dependency."
```

When it tried to use the patch tool to edit a file that didn't exist yet, it didn't panic or loop. It simply caught the error, executed a `write_file` command to create a scratchpad, and successfully re-applied the patch.

But the most absurd moment was how it handled a failure with the `vision_analyze` tool. When the tool threw a 500 error, the agent didn't just pass the error back to me. It paused, introspected its own host engine, and diagnosed the architectural limitation of the server running it:

```
"vision_analyze: 500 'image input not supported - hint: mmproj' -- the local Qwen model has no mmproj (vision projector). Interesting point: I'm
```

literally running on top of that very model." --(paraphrased from trace)

It generated a clean Markdown table summarizing the 8 tools that worked (including SQLite database queries and writing Python scripts to generate images) and accurately diagnosed the root causes for the 4 that failed (like a missing multimodal projector and a misconfigured API gateway).

A 27-billion parameter model, running at 3 bits per weight on a consumer GPU, executing complex tool dependency graphs and debugging its own host environment limits. *That* is why we do this.

Conclusion

If you've made it this far, compiled the binary, configured the tools, and watched a 27B model successfully orchestrate a multi-step workflow on your local machine, the inevitable question hits: *Why not just use Claude Code or a frontier API?*

Let's be brutally honest. If I need a complex script written in five seconds flat, or I want zero-friction, turn-key speed, I am still opening a tab to a hosted model. For most people, most of the time, the sheer convenience of a cloud API wins. The raw truth is that right now, there isn't much in my daily workflow that this local stack *replaces*.

But that is not the point. The differentiators for local inference are conditional, but where they matter, they are absolute.

First, it is free at the margin. You can leave a local agent looping in the background for 72 hours to scrape, parse, and reorganize a massive dataset, and your API bill at the end of the month will still be zero. Second, it is completely sovereign. For a law office that legally cannot paste client files into a cloud provider, or an enterprise dealing with proprietary codebases, a highly capable local 27B model isn't just an alternative—it is the *only* option that exists. Finally, it is resilient. It runs entirely air-gapped. When the internet drops, your workspace doesn't go down with it.

Local inference isn't about beating the massive frontier models at their own game today. It's about the fact that a 27-billion-parameter model can now autonomously run your desktop, debug its own environment, and hold a conversation—all on a 16GB graphics card sitting under your desk. For the tinkerers, the privacy-conscious, and the people building the future, that is more than enough reason to have this stack in your arsenal.

***Addendum: --mmpoj-gpu-swap

buun pointed out I'd skipped one of the nicer features in his fork, and it's worth its own note because it solves a problem you only discover after everything is already working.

Speculative decoding and vision both want VRAM you don't have. The MTP draft context and the vision projector (mmproj) each need their own allocation, and on a 16 or 24 GB card they frequently don't fit at the same time. The usual outcome is an OOM at load, so you pick one: fast, or able to look at pictures.

--mmproj-gpu-swap makes them phase-exclusive instead of concurrent. The projector stays on CPU while you're doing ordinary text work. The moment a request actually contains an image, the server swaps the speculative context out of VRAM, brings the projector onto the GPU for the image phase, then restores the drafter afterward. From server-context.cpp: "Swap the speculative context out so mmproj can use its GPU budget" — and the swap only triggers when a null token (the image placeholder) is found in the prompt. Text-only prompts never pay for it; the code explicitly keeps ordinary prompt scheduling allocation-free.

The part that matters for context length is in the auto-fit path. Rather than reserving space for both, the fitter measures each and keeps the larger reservation — so the context it advertises is one that survives the image phase, instead of one that works until the first screenshot arrives.

buun's numbers on a 24 GB 3090, Qwen3.8-27B UD-Q4_K_XL, MTP + vision together: 161,792 tokens at the default t4 floor, 262,144 at t1. His note on the table is the important bit — "the table doesn't include what MTP + vision without --mmproj-gpu-swap looks like... well, for most of those it OOMs."

Works with DFlash drafters too, not just MTP. One caveat from the source: with an external draft model that isn't reloadable, the server warns "mmproj GPU swap is unavailable for this external draft type; keeping both resident" and falls back to keeping both in memory — so it degrades loudly rather than silently.

From the bottom of my heart, thanks for reading!